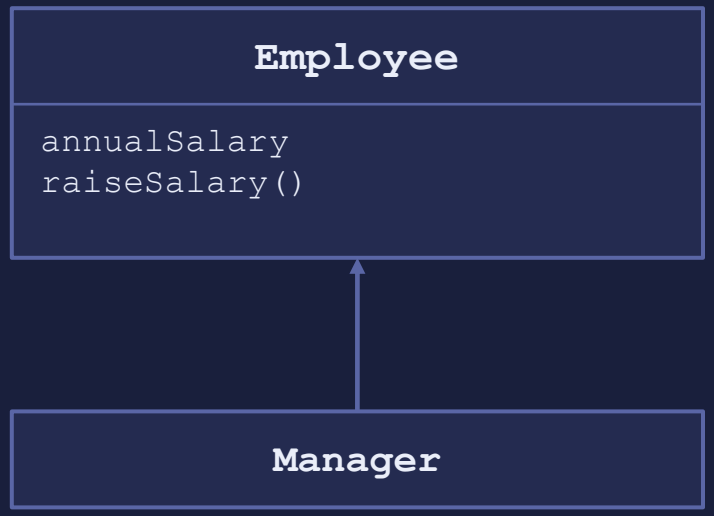
CIS 3300 · INHERITANCE AND POLYMORPHISM

Abstract Classes, final, and Constructors

Part 3 of 4: designing the top of a hierarchy, closing it off, and building objects in order

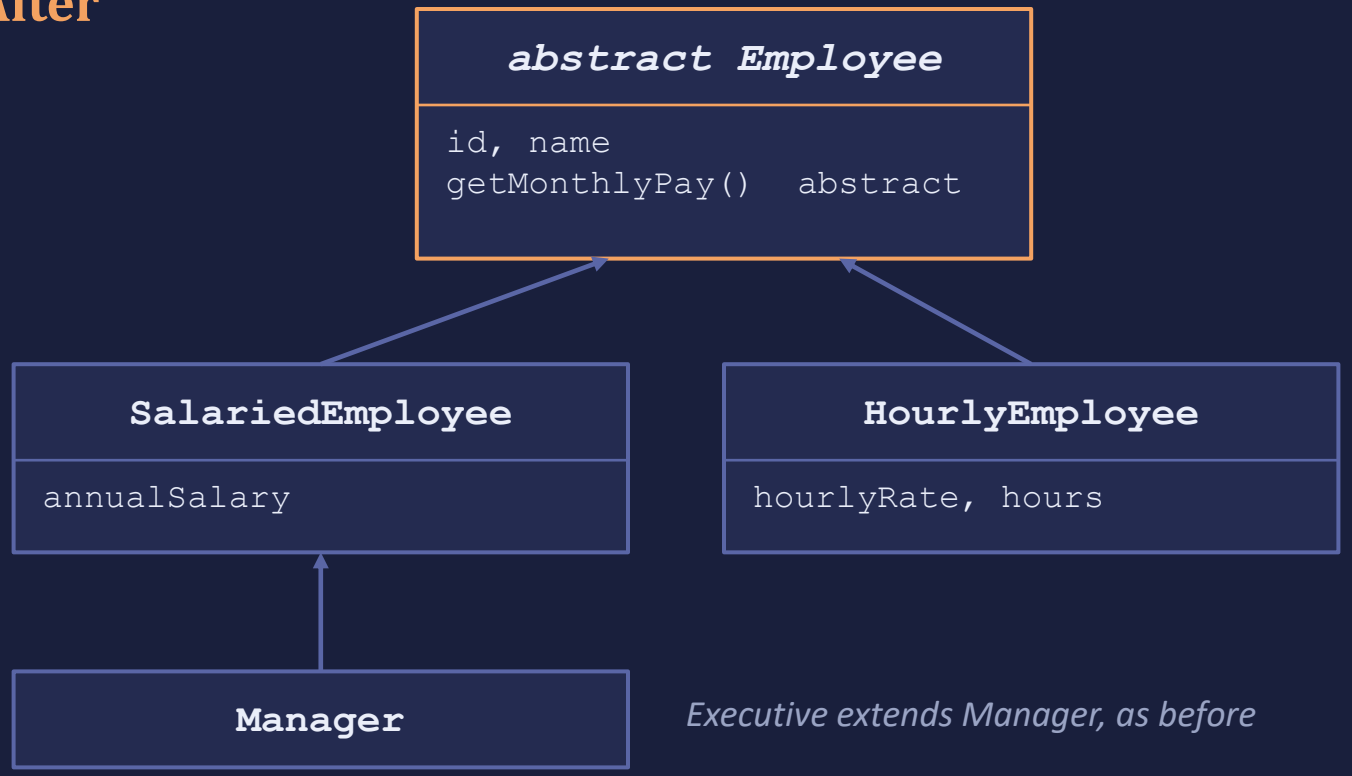
Hourly workers do not fit a salaried Employee

Before



HourlyEmployee? It would inherit a salary.

After



Executive extends Manager, as before

An abstract class is a type you cannot instantiate

```
public abstract class Employee {
    private final String id;
    private String name;

    protected Employee(String name) { ... }  // still has one

    public abstract double getMonthlyPay();   // no body

    public String toString() {   // calls the abstract method
        return id + " " + name + ", " + getMonthlyPay();
    }
}
```

new Employee(...) does not compile: “Employee is abstract; cannot be instantiated.”

Every concrete subclass must supply every abstract method.

It can still have fields, constructors, and concrete methods.

It is still a type: Employee[] and ArrayList<Employee> work as before.

Concrete class: one you can instantiate. An abstract class may have no abstract methods at all.

FINAL

final closes a variable, a method, or a class

Variable or field

Assigned only once. Unrelated to inheritance.

```
private final String id;
```

Method

Cannot be overridden by any subclass.

```
public final boolean equals(...)
```

Class

Cannot be extended at all.

```
public final class Executive
```

Why close something? To protect a rule (equals), to protect immutability (String, records), or because the class was not designed to be extended.

Every constructor calls up the chain first



1. The first statement is super(...) or this(...), never both.
2. Write neither, and Java inserts super(). That fails if the superclass has no no-argument constructor.
3. Write no constructor, and Java supplies a default one. Write any constructor, and the default disappears.
4. Each class initializes only its own fields. Constructors are not inherited and not polymorphic.

Java 25 allows some statements before super(...), as long as they do not use the object being built.

ORDER OF CREATION

Objects are built top-down, one level at a time

```
main starts
Base static block
Derived static block
Base field initializer
Base constructor body
Derived.describe(), label = null
Derived field initializer
Derived constructor body, label = Derived field initializer
```

1. Static blocks: once per class, superclass first.
2. Memory for every field, set to 0, false, or null.
3. The constructor calls up the chain to Object().
4. Back down, per level: field initializers, then the constructor body.

Correction to a common summary: it is not “all field initializers, then all constructors.” Base finishes both before Derived starts its field initializers.

GOTCHA

Never call an overridable method from a constructor

```
Base() {
    describe();   // runs Derived's version
}

class Derived extends Base {
    private String label = "ready";   // not set yet
    @Override void describe() {
        System.out.println(label);   // prints null
    }
}
```

What happens

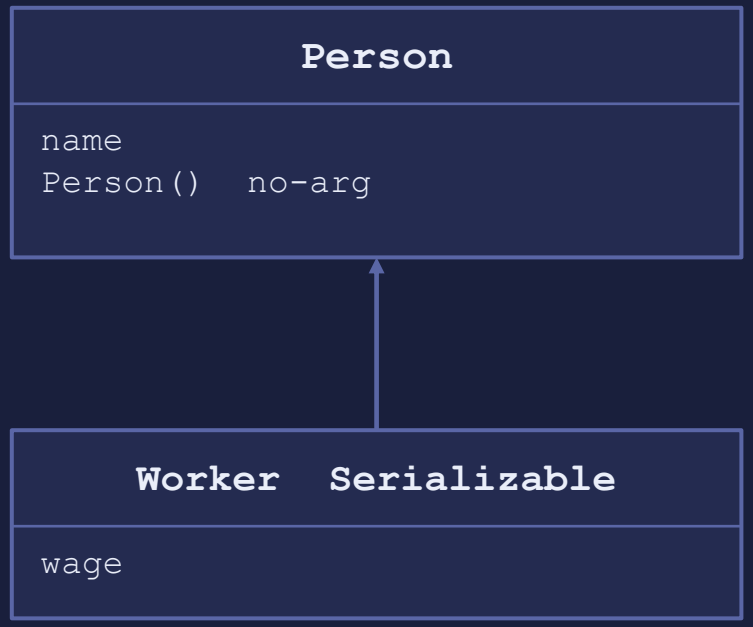
Base runs before Derived's fields are initialized, so the override sees null or 0.

The rule

From a constructor, call only private, static, or final methods.

Constructors that call setters: make those setters final, or share a private helper.

A non-serializable superclass is rebuilt by its constructor




```
Before: Ada earns 31.5
Reading back:
  Person() runs
After:  (unknown) earns 31.5
```

- wage (serializable class) is restored.
- name (non-serializable superclass) is not saved; Person() sets it.
- No accessible no-arg constructor: InvalidClassException, no valid constructor.

If a superclass's fields matter, make the superclass Serializable too.

NEXT

Next: build it in Example 3

1. Make Employee abstract; add SalariedEmployee and HourlyEmployee.
2. Move Manager under SalariedEmployee; make Executive final.
3. Predict the output of ConstructorOrderDemo, then run it.
4. Then: the rental agency problem, which we solve together.

Read the Part 3 notes and Core Java, Volume I, Sections 5.1 and 5.2.